Programming Assignment 3

Eigenfaces for Recognition

CS 479

Jackson Loughmiller

May 4, 2026

I declare that all material in this assignment is my own work except where there is clear acknowledgment or reference to the work of others. I understand that both my report and code may be subjected to plagiarism detection software, and fully accept all consequences if found responsible for plagiarism, as explained in the syllabus, and described in UNR's Academic Standards Policy: UAM 6,502.

Theory

Experiment a: Face Recognition

Dimensionality Reduction

Dimensionality reduction is a technique that is often required in practice due to data often having a large number of dimensions, making it computationally expensive to compute results and requiring large amounts of data for training. Data with very high dimensions incurs the "Curse of Dimensionality", which states that as the amount of dimensions increases, the amount of training data needed increases exponentially. Dimensionality reduction seeks to remedy this problem by using either feature extraction or feature selection, for this report feature extraction is the only relevant method.

Feature Extraction

Feature extraction is a method of dimensionality reduction that creates a new, smaller set of features from the original features by performing certain transformations. This new set of features is related to the original, however they do not correspond to physical features that humans are able to comprehend.

Principle Component Analysis

Principal component analysis (PCA) is a feature extraction technique used for dimensionality reduction. A data point:

$$\mathbf{x} \in \mathbb{R}^D \tag{1}$$

can be represented as a linear combination of an orthonormal set of D basis vectors:

$$\mathbf{x} = \sum_{i=1}^{D} x_i v_i = x_1 v_1 + x_2 v_2 + \ldots + x_D v_D \tag{2}$$

PCA reduces the dimensionality by approximating each vector, x, in a subspace using K«D orthonormal basis vectors:

$$\mathbf{x} = \sum_{i=1}^{K} x_i v_i = x_1 v_1 + x_2 v_2 + \ldots + x_K v_K \tag{3}$$

It does this by minimizing the function below in Eq. 4, conceptually the information loss.

$$\|\mathbf{x} - \hat{\mathbf{x}}\| \tag{4}$$

Overall, PCA consists of essentially 3 steps:

1. Find the eigenvectors of the covariance matrix of the data.
2. Sort the eigenvectors with respect to their eigenvalue, essentially sorting by greatest to smallest variance.
3. Choose the top K eigenvectors, where K is a parameter.

To compute the covariance matrix, one needs to first compute the sample mean, and subtract it from all the data to center it. This can be done using Eqs. 5 and 6, below:

$$\bar{\mathbf{x}} = \frac{1}{M}\sum_{i=1}^{M}\mathbf{x}_i \tag{5}$$

$$\Phi_i = \mathbf{x}_i - \bar{\mathbf{x}} \tag{6}$$

The covariance matrix can then be calculated using Eq. 7, below:

$$\Sigma_{\mathbf{x}} = \frac{1}{M}\sum_{i=1}^{M}(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{M}\sum_{i=1}^{M}\Phi_i\Phi_i^T = \frac{1}{M}AA^T \tag{7}$$

Then, the eigenvectors can be calculated with Eq. 8, below:

$$\Sigma_x u_i = \lambda_i u_i \tag{8}$$

Once the eigenvectors and eigenvalues have been computed, the last step is simply to choose the eigen vectors associated with the top K eigenvalues.

For computational efficiency, Eq. 7 can be calculated approximately using Eq. 9, below:

$$AA^T \tag{9}$$

It can be shown that Eq. 9 gives the top M eigenvectors that would be computed from Eq. 7, and since PCA needs only the top K eigenvectors, this is perfect for this situation.

Results and Discussion

Experiment a

For experiment a, I used the PCA approach to perform facial recognition, using the Mahalanobis distance on the eigenface space projections of query images for classification. Figure 1, below, shows a visual representation of the average face that is subtracted from the images to center the data. The average face does not capture any truly distinct features, which is expected.

Figure 1: Average face computed from the fa_H training set.

Figure 2, below, shows the eigenfaces corresponding to the top 10 largest eigenvalues, and figure 3 shows the eigenfaces corresponding to the smallest 10 eigenvalues.

Figure 2: Top 10 eigenfaces corresponding to the 10 largest eigenvalues.

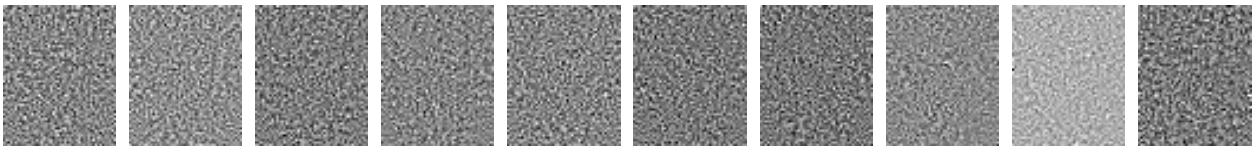


Figure 3: Bottom 10 eigenfaces corresponding to the 10 smallest eigenvalues.

As the figures above show, the top 10 eigenvectors clearly contain more information of the features for a face, showing visible facial-like features in their image representations. Due to lack of containing much feature information, and likely also due to rounding errors in eigenvalues and eigenvectors calculations, the last 10 images contain what appears to be pure noise from the image representations. This makes sense, since the theory behind PCA states the top eigenvectors should contain more information than the lower ones.

Figure 4, below, shows the Cumulative Match Characteristic Curve for retaining 80%, 90%, and 95% of the the data during classification, plotting rank (number of images within which a classification is considered correct), against the accuracy. Interestingly, the model seemed to perform approximately the same at high values of rank for 80% and 90% of the data, however it performed noticeably worse with 95%. This is likely due to the smaller eigenvalue associated eigenvectors containing less information, and potentially being very corrupted with noise from rounding errors. It is notable that with lower rank values (below around 15), both 90% and 95% work better than 80%.

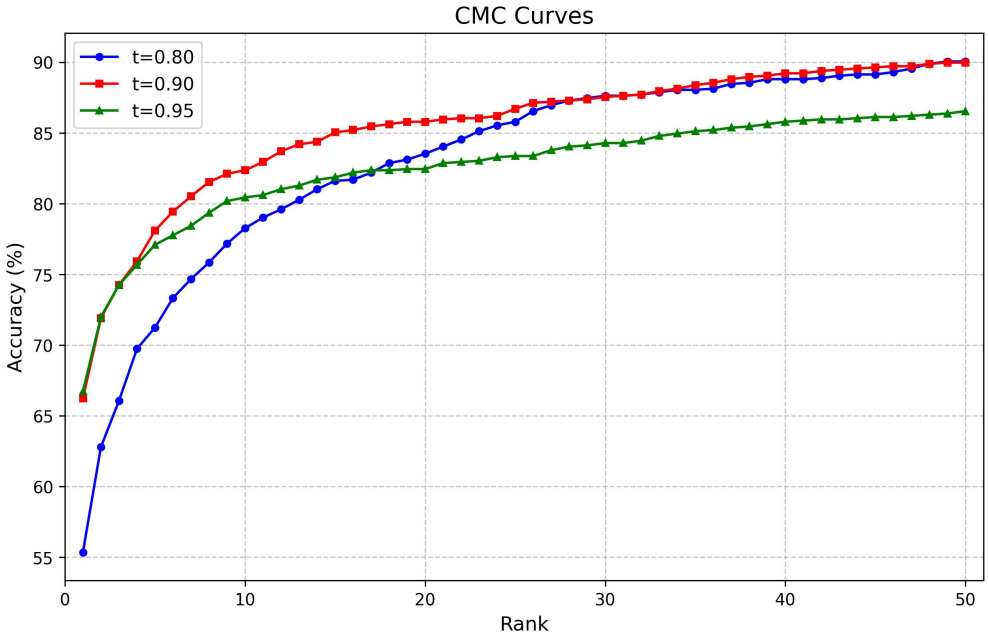
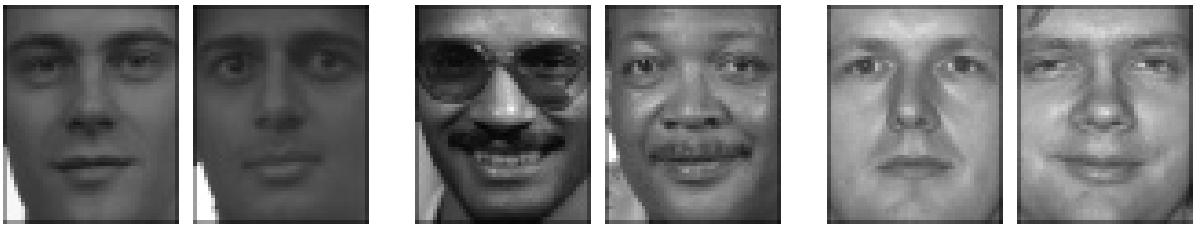


Figure 4: CMC curves comparing 80%, 90%, and 95% variance thresholds (fa_H training, fb_H testing).

Figures 5 - 10 show correctly matched and incorrectly matched pairs for different amounts of information being kept, at 80%, 90%, and 95%.

Query Match Query Match Query Match

Figure 5: Three correctly matched query-gallery pairs at 80% variance threshold (r=1).



Query Mismatch Query Mismatch Query Mismatch

Figure 6: Three incorrectly matched query-gallery pairs at 80% variance threshold (r=1).

As seen by Figures 5 and 6, above, even with the mismatched images, the matches seem to have pretty similar facial features to the query image. Even the pair in which the query image is wearing glasses, the other facial features such as nose shape, mustache, and jawline are fairly aligned.

For both 90% and 95% of the data, the mismatches vary more widely from the features seen in the query images. Some of them appear to be somewhat close at first glance, but upon closer inspection individual features seem much different between query and match.

Query Match Query Match Query Match

Figure 7: Three correctly matched query-gallery pairs at 90% variance threshold (r=1).

Query Mismatch Query Mismatch Query Mismatch

Figure 8: Three incorrectly matched query-gallery pairs at 90% variance threshold (r=1).

For 95% of the data in particular, the features have a lot of variance between query and match in the incorrect matches, Figure 10. In most of the images, at least one prominent feature such as nose shape, eyebrow size, or eye structure are significantly different from the query image.

Query Match Query Match Query Match

Figure 9: Three correctly matched query-gallery pairs at 95% variance threshold (r=1).

Query Mismatch Query Mismatch Query Mismatch

Figure 10: Three incorrectly matched query-gallery pairs at 95% variance threshold (r=1).

These results from the incorrect matches are somewhat unexpected. I expected the 80% information experiment to produces significantly worse, or more noticeably different incorrect matches, than the others based on the data seen in the CMC curve. I suspect that the lower accuracy shown in the CMC curve may be due to features being included that are not very visible to the human eye in the 90% and 95% categories, and are therefore not noticeable in the images.

Experiment b

In experiment b, I tested the models performance dealing with intruders, query images of a face that is not in the database. To do this, I implemented a threshold value, for which if the distance the model output for the Mahalanobis distance was larger than the threshold, the query image would be rejected. Figure 11, below, shows the ROC curve for different values of the threshold. I varied the threshold from 0 - 1.24, the maximum distance the model would output based on the data in the testing set. As seen by the curve, the model outputs more false postives as the threshold increases, which makes sense given the model would be allowing larger distances to reflect a match. True positives increases up until a point, after which the true positives stay relatively the same, while false positives continue to increase. This point likely reflects the tipping point at which the model stops having correct matches with larger distances, and instead is just outputting incorrect matches.

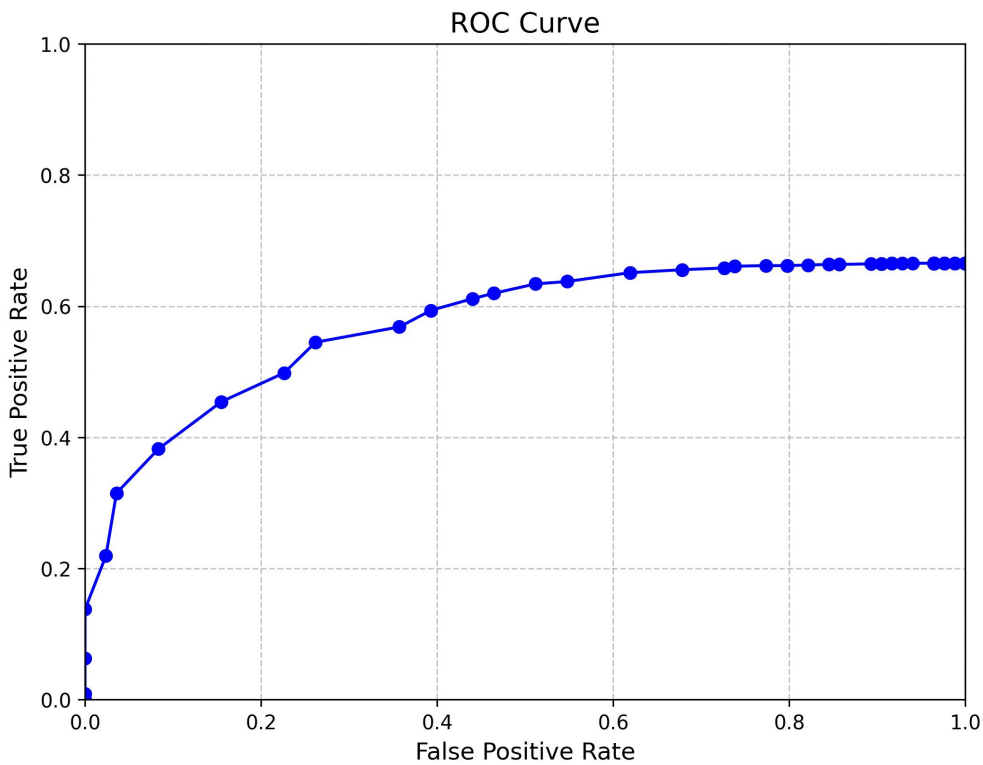


Figure 11: ROC curve for intruder detection (fa2_H training, fb_H testing, 95% variance threshold).